# Rider Payout Dispute Desk

- ProcureYard take-home, ML/AI Engineer Intern.
- Due **Saturday 3 October, 7:30 PM IST**. A 30-minute call follows.
- AI tools allowed. Python or Node.js, any agent framework or none, any LLM provider (the free tiers of Groq and Gemini are enough).

QuickDrop is a fictional delivery company with about 5,000 riders. Its ops team handed us the material below, as it came.

**Email from the Head of Ops**

> Subject: Fwd: rider payout complaints, please fix
>
> Every morning my three executives find hundreds of WhatsApp messages from riders saying their payout is wrong: “bhai kal ka payout 120 kam aaya”, “order T88213 ka surge nahi mila”. They check the trips sheet, work out what the rider should have got, compare it with what we paid, and reply. Two days per reply. Riders are angry, some have quit.
>
> I want an assistant that handles this the way my executives do: it talks to the rider, asks what it needs, checks the trips and payouts, and sorts it out. Riders reply in several messages, change their story, or complain about two things at once. If the rider is right, they get their money and a reply. If they’re wrong, a proper explanation, not silence. Anything unclear or shady comes to my team, and my team needs to see what happened and why. I also need to trust it before it moves money.
>
> Meera

**Note from Finance**

> Auto-pay is fine for small amounts: up to ₹200 per dispute, once per rider per day. Bigger or repeat ones need someone from ops to approve. Never pay more than the rider is owed, we lost money on this last quarter. We reconcile against PaySwift, not against your database.

**Chat from the engineer who set up the messaging vendor**

> the vendor POSTs each rider message to us, one at a time, as the rider types them. if they don’t get a 2xx in ~10s they send it again. wamid is their id for the message, rider_id comes from the phone number it was sent from. whatever we return as reply goes back to the rider.
>
> PaySwift sandbox behaves like production.

**Interface**

```
POST /messages
{ "message_id": "wamid.KAZ5TCF7RARUVH", "rider_id": "R003",
  "text": "Bhai order T926334 ka surge nahi mila, 20 tarikh wala",
  "received_at": "2026-09-22T09:05:00+05:30" }

→ { "reply": "text that goes back to the rider" }

GET /trace/{rider_id}   everything your agent did for this rider, in order
GET /ops/pending        what is waiting for ops: approvals and escalations
```

**What to build**

- An agent that handles rider conversations end to end. You choose its tools, its memory and what the model is allowed to decide.
- Evals for your agent: one command that takes the service URL, runs your test conversations and prints a report. They may use only the interface above and PaySwift, because we will also run them against other implementations.
- A simple page where ops can see conversations and what the agent did, and approve or reject what’s waiting.
- `docker compose up` runs everything: your service, Postgres and the PaySwift sandbox. GitHub Actions run on every push.

**In the repo** (https://github.com/Procureyard/qd-assignment): `docs/policy.md` (ops wiki), `data/` (exports from our systems, plus sample conversations with the end result we’d expect), `payswift/` (provider docs and sandbox). Everything else is yours to decide.

**Submit**

- Clone the repo (don’t fork it), build in a new private repo of your own, add `pratham-saraf` as a collaborator, and reply to this email with the link before the deadline.
- README, one page maximum: how to run it, your agent’s design (its tools, what the model decides and what your code decides), assumptions you made, your eval results, what you skipped.
- The raw exports of your AI sessions in `ai-logs/`, not summaries or screenshots. `SUBMISSION.md` has the logistics.